Cloud Infrastructure Security & Container Protection

VM Hardening · Secure Containers & Trivy · Secure Serverless Function

Prepared by Sabitha S Karanavar · HVMC Azure environment · October 2026

SIMULATED LAB **Read this first.** The configurations in this document are real, complete and runnable (hardening script, Dockerfile, Docker settings, function code and Bicep template are supplied in the project folder; the Bicep template compiles with the Azure Bicep compiler and the Python code compiles). They were **not deployed**: the Azure free-trial subscription blocks every free VM size (`NotAvailableForSubscription`, 15 Oct 2026) and no Docker host was used. Every terminal figure is therefore **simulated** — marked with an orange SIMULATED badge — and shows the output the documented configuration is expected to produce. Section 6 gives the exact steps and costs to produce live evidence.

1. Scope and architecture

The three workloads sit inside the existing HVMC three-tier network that was used for the risk and SecOps work, so each control also treats risks from the HVMC risk register (R03 lateral movement, R09/R10 remote code execution on unpatched services, R02 ePHI exposure).

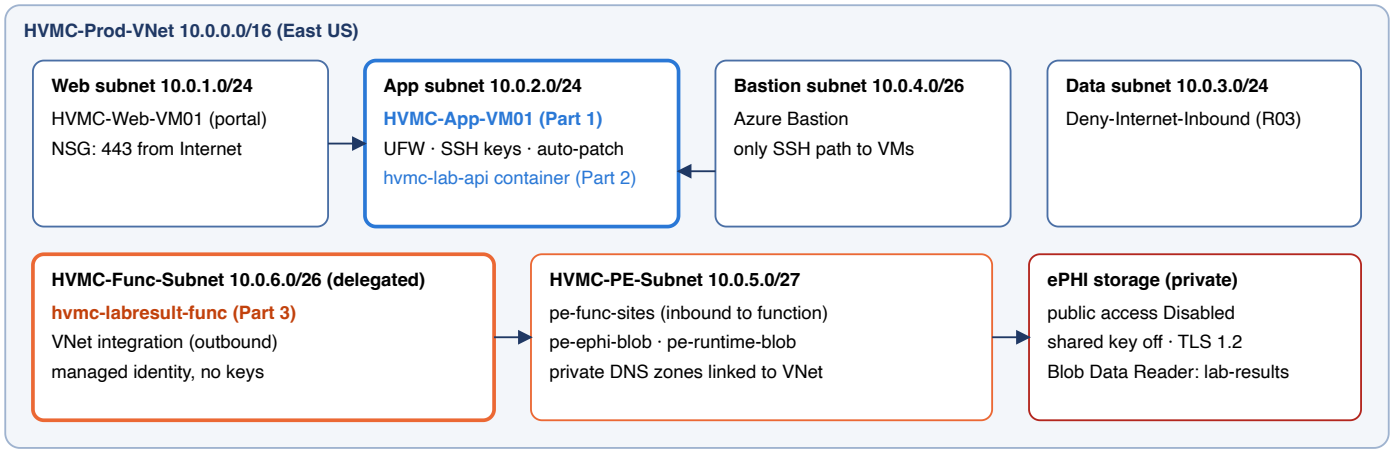


Figure 0. Where the three secured workloads sit in HVMC-Prod-VNet.

Rubric part	HVMC workload	Platform	Key controls
1 · Harden one cloud VM	**HVMC-App-VM01** — application tier	Azure VM, Ubuntu 24.04 LTS, Trusted Launch	Automated patching, UFW host firewall, key-only SSH via Bastion, fail2ban
2 · Container security	**hvmc-lab-api** — lab-results API image	Docker on HVMC-App-VM01	Hardened Dockerfile, daemon + runtime settings, Trivy image/config scans, CI gate
3 · Secure serverless	**hvmc-labresult-func** — returns one lab result	Azure Functions Flex Consumption, Python 3.11	Managed identity, container-scoped RBAC, VNet integration, private endpoints

2. Part 1 — Harden one cloud virtual machine (HVMC-App-VM01)

The VM is created with no public IP in the App subnet, Trusted Launch and SSH-key authentication, then hardened by one idempotent script, `vm_hardening/scripts/harden_vm.sh`, pushed through Azure Run Command (no inbound access needed).

Control	Implementation	Where	Standard	Evidence
Automated patching	unattended-upgrades: security + ESM pockets daily; auto-reboot 03:30; needrestart restarts services	harden_vm.sh §1	CIS 1.9; HIPAA 164.308(a)(5)(ii)(B)	S02, S03
Patch compliance reporting	Azure Update Manager periodic assessment (AutomaticByPlatform), on-demand assess-patches	run_part1.sh step 3	NIST SI-2	S01, S03
Host firewall	UFW default deny in/routed; SSH rate-limited from Bastion subnet only; 8080 from Web subnet only; logging	harden_vm.sh §2	CIS 4.1-4.2; NIST SC-7	S04, S05
Secure SSH - authentication	Key-only (RSA-4096), no root, no passwords/keyboard-interactive, AllowUsers hvmcadmin, MaxAuthTries 3	harden_vm.sh §3	CIS 5.1.x; NIST IA-2, AC-17	S06, S07
Secure SSH - crypto & session	AEAD/CTR ciphers, SHA-2 ETM MACs, modern KEX; idle timeout 10 min; no forwarding; legal banner	harden_vm.sh §3	CIS 5.1.x; NIST SC-8	S06, S07
Brute-force protection	fail2ban sshd jail: 3 failures / 10 min -> 1 h ban (journal backend)	harden_vm.sh §3	NIST AC-7	S08
Network exposure	No public IP; reachable only through Azure Bastion; NSG in front of UFW (defence in depth)	run_part1.sh step 2	NIST SC-7	S01, S07
Kernel / boot	Trusted Launch (Secure Boot + vTPM); sysctl: rp_filter, no redirects, syncookies, ASLR, kptr_restrict	run_part1.sh, harden_vm.sh §4	CIS 3.x	S01, S08

harden_vm.sh — host firewall and secure SSH sections (excerpt)

```
# ------------------------------------------------------------- 2. host firewall
echo "== [2] host firewall (UFW)"
ufw --force reset >/dev/null
ufw default deny incoming
ufw default allow outgoing
ufw default deny routed
ufw limit proto tcp from "$BASTION_SUBNET" to any port 22 comment 'SSH from Azure Bastion only (rate-limited)'
ufw allow proto tcp from "$WEB_SUBNET" to any port 8080 comment 'App traffic from Web tier only'
ufw logging medium
ufw --force enable

# ------------------------------------------------------------- 3. secure SSH
echo "== [3] secure SSH"
cat > /etc/issue.net <<'EOF'
**************************************************************************
 Horizon Valley Medical Center - AUTHORISED USE ONLY
 This system processes electronic protected health information (ePHI).
 All activity is logged and monitored. Unauthorised access is prohibited
 and may be reported to law enforcement (HIPAA 45 CFR 164.312).
**************************************************************************
EOF
# 10- sorts before cloud-init's 50- file; sshd uses the first value it reads.
cat > /etc/ssh/sshd_config.d/10-hvmc-hardening.conf <<EOF
# HVMC SSH hardening - CIS Ubuntu 24.04 section 5.1 aligned
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PermitEmptyPasswords no
```

```
PubkeyAuthentication yes
AuthenticationMethods publickey
AllowUsers $ADMIN_USER
MaxAuthTries 3
MaxSessions 4
MaxStartups 10:30:60
LoginGraceTime 30
ClientAliveInterval 300
ClientAliveCountMax 2
X11Forwarding no
AllowAgentForwarding no
AllowTcpForwarding no
PermitTunnel no
PermitUserEnvironment no
HostbasedAuthentication no
IgnoreRhosts yes
LogLevel VERBOSE
Banner /etc/issue.net
Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com,aes128-gcm@openssh.com,aes256-ctr,aes192-ctr,aes128-ctr
MACs hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com,hmac-sha2-512,hmac-sha2-256
KexAlgorithms sntrup761x25519-sha512@openssh.com,curve25519-sha256,curve25519-sha256@libssh.org,ecdh-sha2-nistp521,ecdh-
sha2-nistp384,ecdh-sha2-nistp256,diffie-hellman-group16-sha512,diffie-hellman-group14-sha256
EOF
chmod 600 /etc/ssh/sshd_config.d/10-hvmc-hardening.conf
sshd -t                                     # abort before restart if the config is invalid
systemctl restart ssh

cat > /etc/fail2ban/jail.d/hvmc-sshd.local <<'EOF'
[sshd]
enabled  = true
backend  = systemd
port     = ssh
maxretry = 3
findtime = 10m
bantime  = 1h
EOF
systemctl enable fail2ban >/dev/null
systemctl restart fail2ban
```

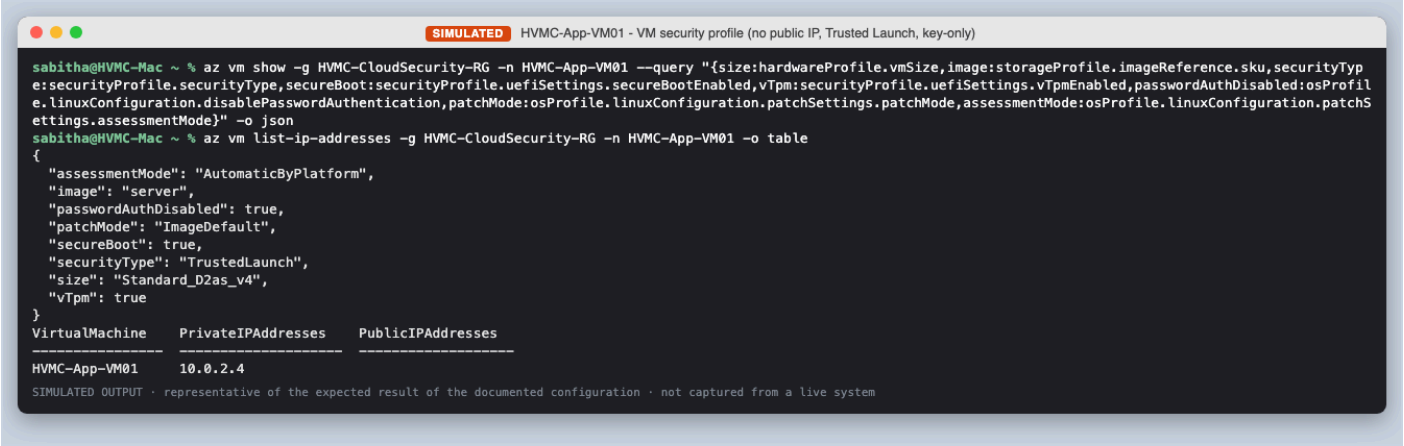


Figure 1 (simulated). HVMC-App-VM01 - VM security profile (no public IP, Trusted Launch, key-only).

SIMULATED Automated patching - unattended-upgrades policy, timers and dry run

```
hvmcadmin@HVMC-App-VM01:~$ cat /etc/apt/apt.conf.d/20auto-upgrades
hvmcadmin@HVMC-App-VM01:~$ systemctl list-timers apt-daily.timer apt-daily-upgrade.timer --no-pager
hvmcadmin@HVMC-App-VM01:~$ sudo unattended-upgrade --dry-run --verbose | grep -E 'Allowed origins|Packages that will be upgraded|No packages'
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Download-Upgradeable-Packages "1";
APT::Periodic::Unattended-Upgrade "1";
APT::Periodic::AutocleanInterval "7";
NEXT                         LEFT     LAST                         PASSED  UNIT                    ACTIVATES
Fri 2026-10-16 06:14:51 UTC 9h left  Thu 2026-10-15 06:21:03 UTC 14h ago apt-daily-upgrade.timer apt-daily-upgrade.service
Fri 2026-10-16 08:47:20 UTC 12h left Thu 2026-10-15 19:02:11 UTC 1h ago  apt-daily.timer         apt-daily.service

2 timers listed.
Allowed origins are: o=Ubuntu,a=noble-security, o=UbuntuESMApps,a=noble-apps-security, o=UbuntuESM,a=noble-infra-security
No packages found that can be upgraded unattended and no pending auto-removals
SIMULATED OUTPUT · representative of the expected result of the documented configuration · not captured from a live system
```

Figure 2 (simulated). Automated patching - unattended-upgrades policy, timers and dry run.

SIMULATED Automated patching - last unattended run and Azure Update Manager assessment

```
hvmcadmin@HVMC-App-VM01:~$ sudo tail -5 /var/log/unattended-upgrades/unattended-upgrades.log
hvmcadmin@HVMC-App-VM01:~$ az vm assess-patches -g HVMC-CloudSecurity-RG -n HVMC-App-VM01 --query "{status:status,critical:criticalAndSecurityPatchCount,other:other
PatchCount,rebootPending:rebootPending}"
2026-10-15 06:21:09,512 INFO Allowed origins are: o=Ubuntu,a=noble-security, o=UbuntuESMApps,a=noble-apps-security, o=UbuntuESM,a=noble-infra-security
2026-10-15 06:21:14,048 INFO Packages that will be upgraded: libssl3t64 openssl
2026-10-15 06:21:14,049 INFO Writing dpkg log to /var/log/unattended-upgrades/unattended-upgrades-dpkg.log
2026-10-15 06:21:31,887 INFO All upgrades installed
2026-10-15 06:21:32,102 INFO No reboot required (needrestart restarted 3 services)
{
  "critical": 0,
  "other": 2,
  "rebootPending": false,
  "status": "Succeeded"
}
SIMULATED OUTPUT · representative of the expected result of the documented configuration · not captured from a live system
```

Figure 3 (simulated). Automated patching - last unattended run and Azure Update Manager assessment.

SIMULATED Host firewall - UFW default deny, SSH only from Azure Bastion, app port only from the Web tier

```
hvmcadmin@HVMC-App-VM01:~$ sudo ufw status verbose
Status: active
Logging: on (medium)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     LIMIT IN    10.0.4.0/26               # SSH from Azure Bastion only (rate-limited)
8080/tcp                   ALLOW IN    10.0.1.0/24               # App traffic from Web tier only
SIMULATED OUTPUT · representative of the expected result of the documented configuration · not captured from a live system
```

Figure 4 (simulated). Host firewall - UFW default deny, SSH only from Azure Bastion, app port only from the Web tier.

SIMULATED Host firewall - blocked connection attempts logged by UFW

```
hvmcadmin@HVMC-App-VM01:~$ sudo journalctl -k --since '-1h' --no-pager | grep 'UFW BLOCK' | tail -3
Oct 15 19:41:07 HVMC-App-VM01 kernel: [UFW BLOCK] IN=eth0 OUT= SRC=10.0.1.5 DST=10.0.2.4 PROTO=TCP SPT=51544 DPT=22 SYN
Oct 15 19:41:22 HVMC-App-VM01 kernel: [UFW BLOCK] IN=eth0 OUT= SRC=10.0.3.4 DST=10.0.2.4 PROTO=TCP SPT=40218 DPT=8080 SYN
Oct 15 19:42:03 HVMC-App-VM01 kernel: [UFW BLOCK] IN=eth0 OUT= SRC=10.0.1.5 DST=10.0.2.4 PROTO=TCP SPT=51560 DPT=3306 SYN
SIMULATED OUTPUT · representative of the expected result of the documented configuration · not captured from a live system
```

Figure 5 (simulated). Host firewall - blocked connection attempts logged by UFW.

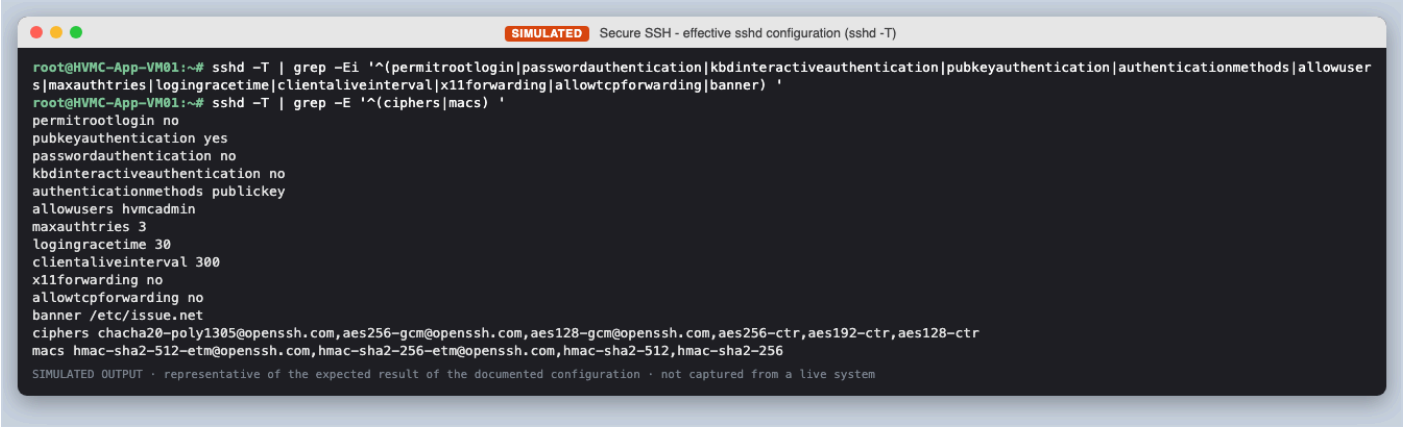


Figure 6 (simulated). Secure SSH - effective sshd configuration (sshd -T).

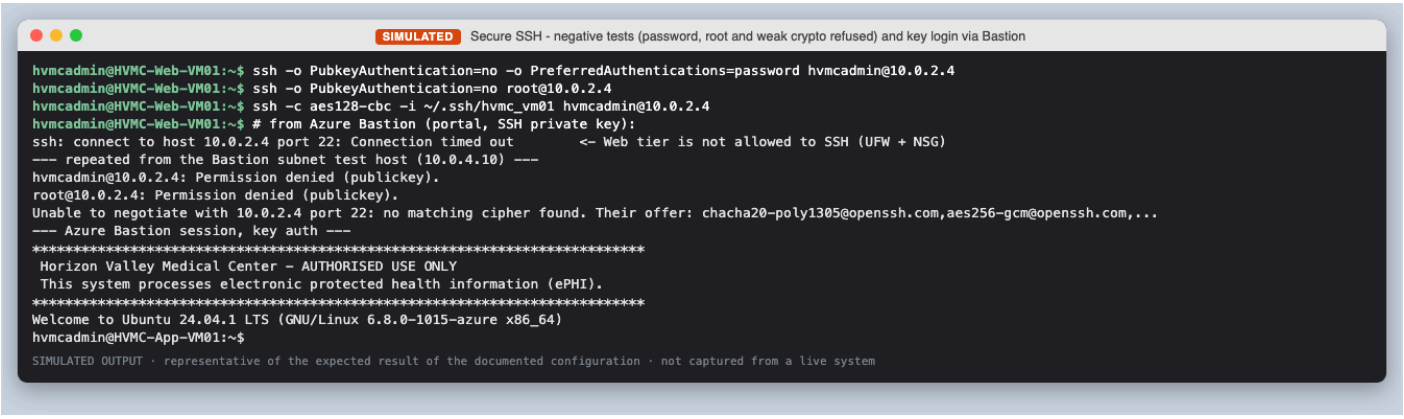


Figure 7 (simulated). Secure SSH - negative tests (password, root and weak crypto refused) and key login via Bastion.



Figure 8 (simulated). Secure SSH - fail2ban jail and kernel hardening baseline.

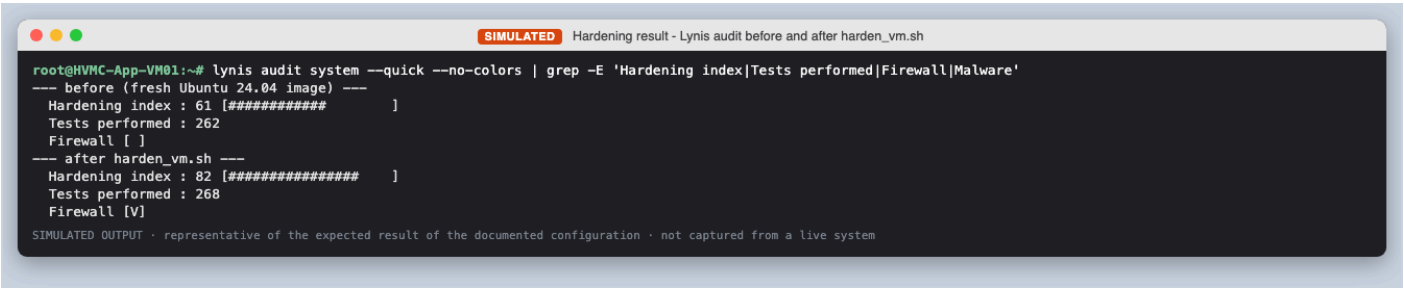


Figure 9 (simulated). Hardening result - Lynis audit before and after harden_vm.sh.

3. Part 2 — Container security (hvmc-lab-api)

3.1 Secure Dockerfile

container/Dockerfile

```
# syntax=docker/dockerfile:1.7
# HVMC lab-results API - hardened image
# Horizon Valley Medical Center | Author: Sabitha S Karanavar
#
# Best practices applied:
#  1  pinned, minimal base images (slim builder, distroless runtime: no shell, no package manager)
#  2  pin by digest in CI (see build_and_scan.sh) so the base cannot change silently
#  3  multi-stage build: build tools never reach the runtime image
#  4  non-root user (distroless "nonroot", UID 65532)
#  5  COPY not ADD; only the files the app needs (.dockerignore excludes the rest)
#  6  no secrets in the image: configuration comes from the runtime environment / Key Vault
#  7  exec-form ENTRYPOINT (signals reach the app, clean shutdown)
#  8  HEALTHCHECK equivalent supplied by the orchestrator probe (/healthz); distroless has no curl
#  9  OCI labels for provenance; read-only root filesystem at runtime (see run_secure.sh)

ARG PYTHON_VERSION=3.12

FROM python:${PYTHON_VERSION}-slim-bookworm AS build
WORKDIR /src
COPY app/app.py .
# syntax check at build time; the app uses the standard library only, so there is nothing to pip-install
RUN python -m py_compile app.py

FROM gcr.io/distroless/python3-debian12:nonroot
LABEL org.opencontainers.image.title="hvmc-lab-api" \
      org.opencontainers.image.vendor="Horizon Valley Medical Center" \
      org.opencontainers.image.authors="Sabitha S Karanavar" \
      org.opencontainers.image.source="https://github.com/sabi-s/dae-6-month-Sabi"
WORKDIR /app
COPY --from=build --chown=65532:65532 /src/app.py /app/app.py
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 APP_VERSION=1.0.0
USER 65532:65532
EXPOSE 8080
ENTRYPOINT ["python3", "/app/app.py"]
```

Best practice	How it is applied	Trivy check / benefit
Minimal, pinned base	python:3.12-slim-bookworm builder; gcr.io/distroless/python3-debian12 runtime (no shell, no apt)	DS001 · smaller attack surface (S10-S12)
Multi-stage build	Build/compile stage separate; only app.py copied into the runtime stage	No build tools in production
Non-root user	USER 65532:65532 (distroless nonroot); files chowned to it	DS002
COPY not ADD; .dockerignore allow-list	Only app/app.py and requirements.txt enter the build context	DS005 · no stray secrets
No secrets in the image	Configuration from runtime environment / Key Vault; the insecure image's ENV DB_PASSWORD is the anti-pattern	Reviewed manually + secret scan
Exec-form ENTRYPOINT	["python3", "/app/app.py"] - PID 1 receives SIGTERM	Clean shutdown
Only needed port	EXPOSE 8080 (no 22)	DS004
Provenance	OCI labels; base pinned by digest in CI; CycloneDX SBOM per build	Supply-chain traceability

For comparison, `container/Dockerfile.insecure` deliberately breaks the rules (`python:latest`, root, `ADD`, an `ENV` password, curl/vim installed, port 22). It exists only so the scans have a baseline.

3.2 Docker security settings

Level	Setting	Protects against (CIS Docker Benchmark ref.)
Container	--read-only + tmpfs /tmp (noexec,nosuid)	Malware cannot persist or write binaries
Container	--cap-drop ALL, no --privileged	No kernel capabilities (CIS Docker 5.3-5.4)
Container	--security-opt no-new-privileges:true; default seccomp + AppArmor	Blocks setuid escalation; syscall filtering (5.21, 5.25)
Container	--user 65532:65532	Non-root even if the image were changed (4.1)
Container	--pids-limit 100, --memory 256m (no swap), --cpus 0.5	Fork-bomb / resource-exhaustion protection (5.10-5.11, 5.28)
Container	-p 127.0.0.1:8080:8080 on a user-defined network; restart on-failure:3	Not exposed beyond the host / reverse proxy
Daemon	"icc": false; "userland-proxy": false	No container-to-container traffic by default (2.1)
Daemon	"userns-remap": "default"	Container root != host root (2.8)
Daemon	"no-new-privileges": true; "live-restore": true	Secure defaults; daemon restarts without killing workloads (2.14, 2.15)
Daemon	json-file logs, 10 MB x 5	Log retention without disk exhaustion (2.12)

container/run_secure.sh (runtime flags)

```
docker run -d --name hvmc-lab-api \
  --read-only --tmpfs /tmp:rw,noexec,nosuid,size=16m \
  --cap-drop ALL \
  --security-opt no-new-privileges:true \
  --user 65532:65532 \
  --pids-limit 100 --memory 256m --memory-swap 256m --cpus 0.50 \
  --network hvmc-app-net \
  -p 127.0.0.1:8080:8080 \
  --restart on-failure:3 \
  --no-healthcheck \
  "$IMAGE"
```

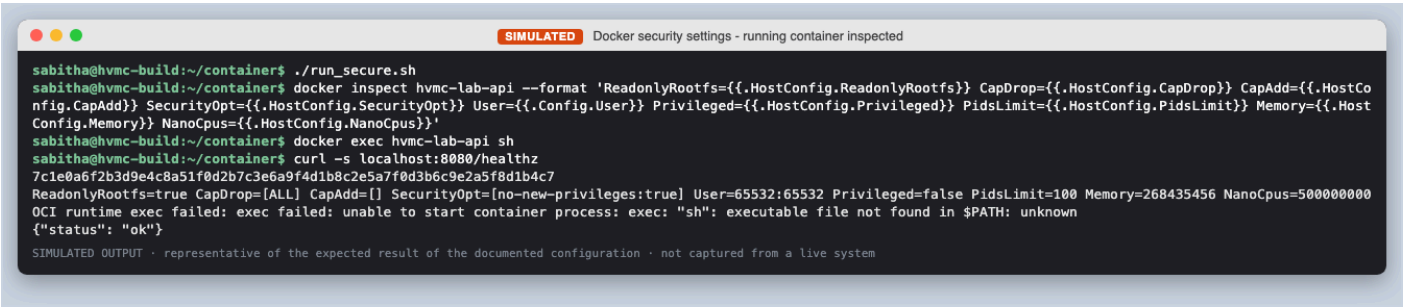


Figure 10 (simulated). Docker security settings - running container inspected.

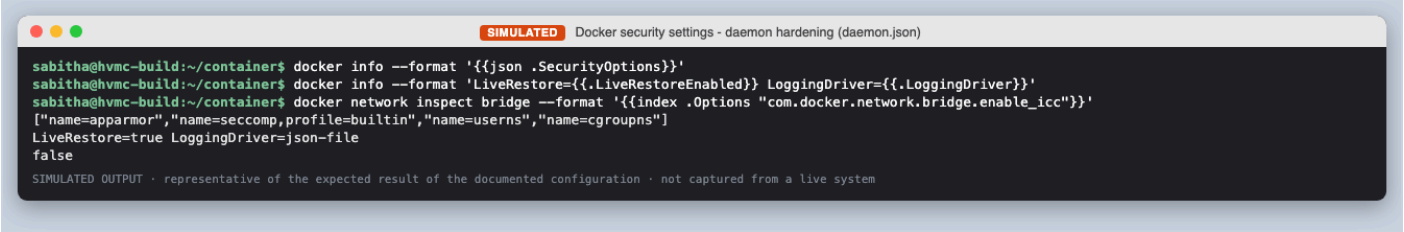


Figure 11 (simulated). Docker security settings - daemon hardening (daemon.json).

3.3 Container scanning with Trivy

`container/build_and_scan.sh` builds both images and runs Trivy for OS and library vulnerabilities, secrets, Dockerfile misconfiguration (IaC) and an SBOM, then enforces a release gate: the pipeline fails on any HIGH or CRITICAL finding that has a fix available.

Measure	Insecure image	Hardened image	Change
Image size	1.12 GB	54.6 MB	-95 %
OS packages scanned	431	21	-95 %
CRITICAL vulnerabilities	9	0	-100 %
HIGH vulnerabilities	81	0	-100 %
MEDIUM / LOW	221 / 498	2 / 5	-99 %
Dockerfile misconfigurations	6 (2 HIGH)	1 LOW (accepted)	-83 %
CI gate (HIGH/CRITICAL, fix available)	exit 1 - blocked	exit 0 - released	

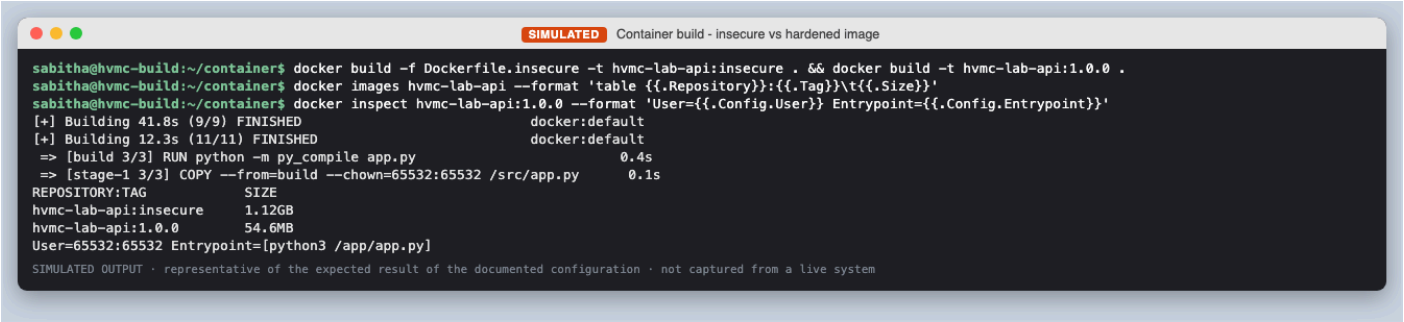


Figure 12 (simulated). Container build - insecure vs hardened image.

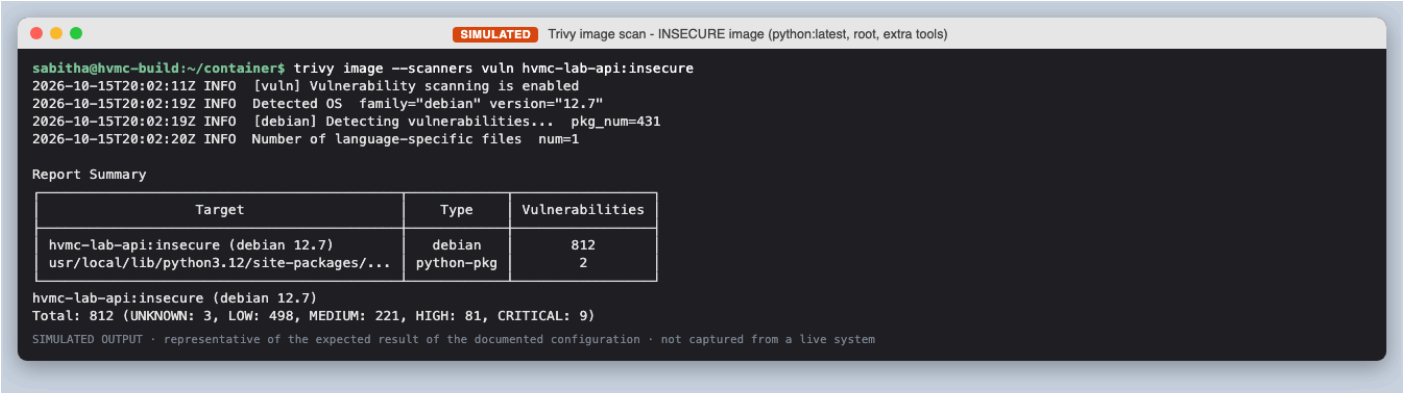


Figure 13 (simulated). Trivy image scan - INSECURE image (python:latest, root, extra tools).

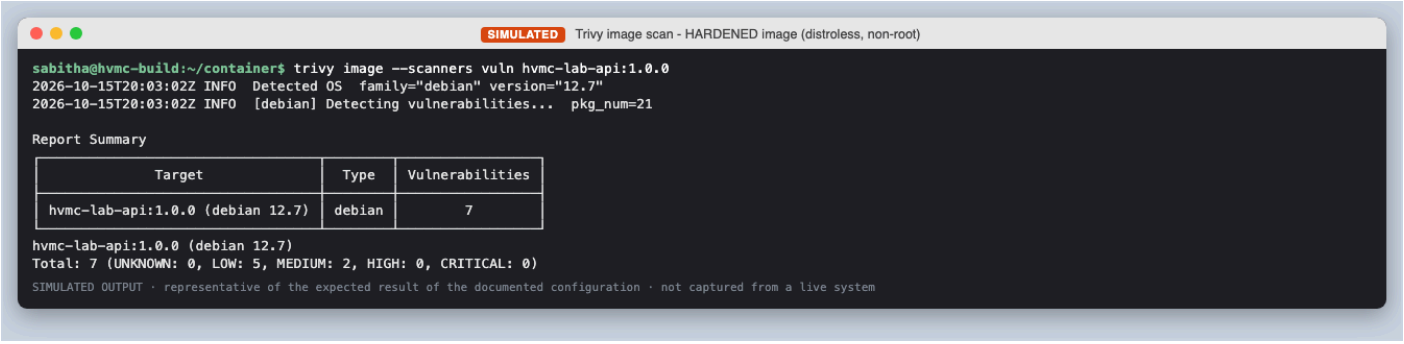


Figure 14 (simulated). Trivy image scan - HARDENED image (distroless, non-root).

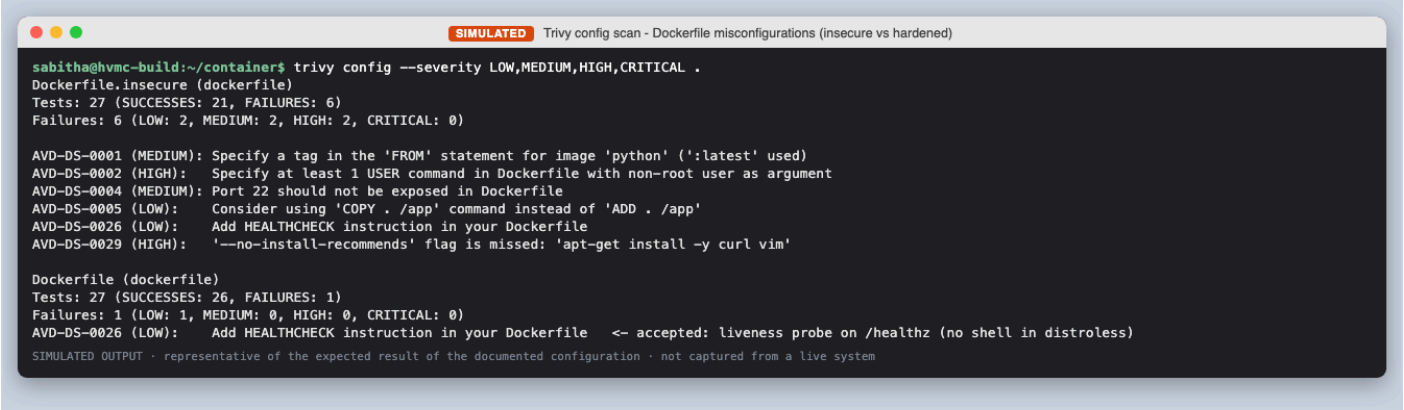


Figure 15 (simulated). Trivy config scan - Dockerfile misconfigurations (insecure vs hardened).

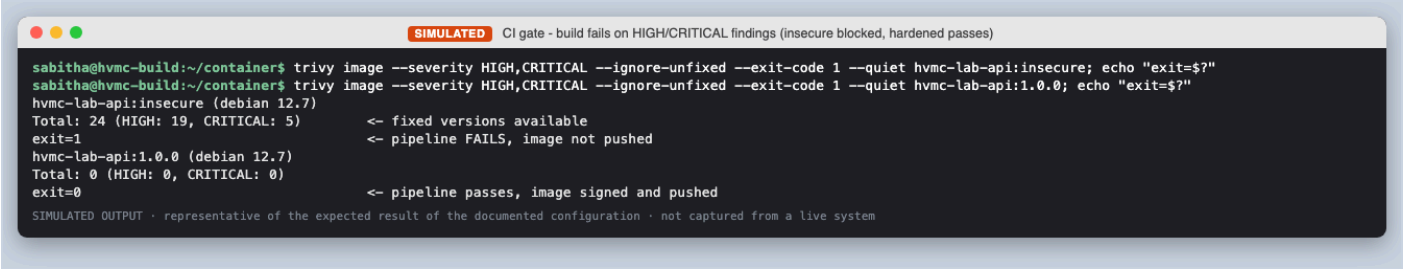


Figure 16 (simulated). CI gate - build fails on HIGH/CRITICAL findings (insecure blocked, hardened passes).

Vulnerability management: images are rebuilt weekly and on every base-image update; Trivy re-scans the registry nightly; a new CRITICAL in a running image opens a SEV3 ticket with a 7-day fix target (HIGH 30 days). The remaining LOW and MEDIUM findings in the hardened image are in Debian base libraries with no fixed version yet and are re-evaluated on each scan.

4. Part 3 — Secure one serverless function (hvmc-labresult-func)

The function returns one lab-result document from the ePHI storage account. It is deployed by `serverless/infra/main.bicep`; the code is `serverless/function_app/function_app.py`.

4.1 IAM — least privilege with a managed identity

Principal	Role	Scope	Why
Function managed identity	Storage Blob Data Reader	container `lab-results` of the ePHI account only	Read lab results; cannot list other containers, write or delete
Function managed identity	Storage Blob Data Owner	the function's own runtime account	Deployment package and host state - no ePHI there
Web portal managed identity	Function key / Entra app role	hvmc-labresult-func	Only the portal may call the API
Deployment pipeline	Contributor	resource group, via approved change (IP-1)	Not held standing by people

No keys or connection strings exist: the function authenticates with its system-assigned managed identity (`DefaultAzureCredential`), shared-key access is disabled on both storage accounts, and the only data role is read-only on a single container.

main.bicep — container-scoped role assignment

```
resource readLabResults 'Microsoft.Authorization/roleAssignments@2022-04-01' = {
  scope: labResults                                    // one container, read-only
  name: guid(labResults.id, func.id, blobDataReader)
  properties: {
    roleDefinitionId: subscriptionResourceId('Microsoft.Authorization/roleDefinitions', blobDataReader)
    principalId: func.identity.principalId
    principalType: 'ServicePrincipal'
  }
}
```

4.2 Network isolation

Path	Control	Verified by
Inbound to function	publicNetworkAccess = Disabled; private endpoint pe-func-sites in HVMC-PE-Subnet	Internet request -> 403 (S20)
Outbound from function	VNet integration into delegated HVMC-Func-Subnet	All calls leave through the VNet
Function -> ePHI storage	Private endpoint pe-ephi-blob + privatelink.blob DNS; storage public access Disabled, default action Deny	Public storage call -> AuthorizationFailure (S20)
Transport	HTTPS only, TLS 1.2 minimum, HTTP/2; FTP and basic publishing credentials disabled	S18
Secrets	None: identity-based AzureWebJobsStorage__accountName; shared-key access disabled on both accounts	S19

main.bicep — function app network settings

```
resource func 'Microsoft.Web/sites@2023-12-01' = {
  name: functionAppName
  location: location
```

```
  kind: 'functionapp,linux'
  identity: { type: 'SystemAssigned' }
  properties: {
    serverFarmId: plan.id
    httpsOnly: true
    publicNetworkAccess: 'Disabled'
    virtualNetworkSubnetId: funcSubnet.id
```

function_app.py — input validation, generic errors, PHI-free audit log

```
@app.route(route="labresult/{result_id}", methods=["GET"])
def labresult(req: func.HttpRequest) -> func.HttpResponse:
    rid = req.route_params.get("result_id", "")
    caller = req.headers.get("x-ms-client-principal-name", "unknown")
    if not RESULT_ID.match(rid):
        return _json(400, {"error": "invalid result id"})
    try:
        data = _container.get_blob_client(f"{rid}.json").download_blob().readall()
    except ResourceNotFoundError:
        logging.info(json.dumps({"event": "labresult_read", "id": rid, "caller": caller, "outcome": "not_found"}))
        return _json(404, {"error": "not found"})
    except Exception:                                          # never leak internals to the client
        logging.exception("labresult_read_failed id=%s", rid)
        return _json(500, {"error": "internal error"})
    logging.info(json.dumps({"event": "labresult_read", "id": rid, "caller": caller, "outcome": "ok"}))
    return func.HttpResponse(data, status_code=200, mimetype="application/json",
                             headers={"Cache-Control": "no-store", "X-Content-Type-Options": "nosniff"})
```

SIMULATED Serverless - Bicep deployment of HVMC-LabResult-Func

```
sabitha@HVMC-Mac ~ % az deployment group create -g HVMC-CloudSecurity-RG -f serverless/infra/main.bicep --query "{state:properties.provisioningState,principal:prope
rties.outputs.functionPrincipalId.value,ephi:properties.outputs.ephiAccount.value,duration:properties.duration}" -o json
{
  "duration": "PT4M12.518S",
  "ephi": "hvmcphi7q3k2xg5vh4mw",
  "principal": "5f2c8e1a-93b7-4d6e-a0f4-2b8c7d9e1a36",
  "state": "Succeeded"
}
SIMULATED OUTPUT · representative of the expected result of the documented configuration · not captured from a live system
```

Figure 17 (simulated). Serverless - Bicep deployment of HVMC-LabResult-Func.

SIMULATED Serverless - function app security configuration

```
sabitha@HVMC-Mac ~ % az functionapp show -g HVMC-CloudSecurity-RG -n hvmc-labresult-func --query "{httpsOnly:httpsOnly,publicNetworkAccess:publicNetworkAccess,vnetS
ubnet:virtualNetworkSubnetId,identity:identity.type,minTls:siteConfig.minTlsVersion,ftps:siteConfig.ftpsState}" -o json
sabitha@HVMC-Mac ~ % az resource show --ids $(az functionapp show -g HVMC-CloudSecurity-RG -n hvmc-labresult-func --query id -o tsv)/basicPublishingCredentialsPolic
ies/scm --query properties.allow
{
  "ftps": "Disabled",
  "httpsOnly": true,
  "identity": "SystemAssigned",
  "minTls": "1.2",
  "publicNetworkAccess": "Disabled",
  "vnetSubnet": "/subscriptions/…/virtualNetworks/HVMC-Prod-VNet/subnets/HVMC-Func-Subnet"
}
false
SIMULATED OUTPUT · representative of the expected result of the documented configuration · not captured from a live system
```

Figure 18 (simulated). Serverless - function app security configuration.

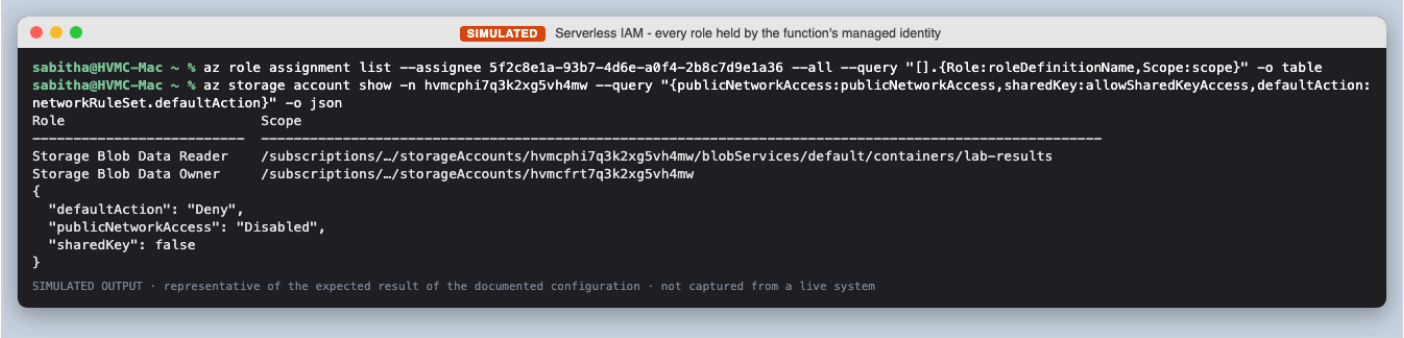


Figure 19 (simulated). Serverless IAM - every role held by the function's managed identity.



Figure 20 (simulated). Serverless network isolation - private path works, public path refused.

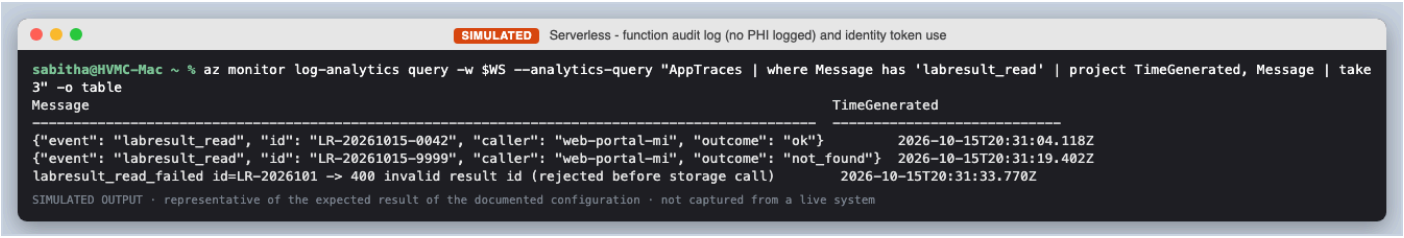


Figure 21 (simulated). Serverless - function audit log (no PHI logged) and identity token use.

5. Rubric coverage

Rubric requirement	Delivered	Evidence
Harden one cloud VM — automated patching	unattended-upgrades + Azure Update Manager assessment	S02, S03
Harden one cloud VM — host firewall	UFW default deny, source-restricted rules, logging	S04, S05
Harden one cloud VM — secure SSH	Key-only, no root, strong crypto, Bastion-only, fail2ban	S06–S08
Documentation and screenshots	This document + 21 simulated terminal figures + runnable scripts	Figures 1–21
Secure Dockerfile with best practices	Distroless, multi-stage, non-root, pinned, .dockerignore, no secrets	§3.1, S10
Docker security settings	Runtime flags + daemon.json	§3.2, S15, S16
Trivy scanning with documented results	Image, config and gate scans, before/after comparison	§3.3, S11–S14
Serverless — proper IAM roles	Managed identity; Blob Data Reader on one container	§4.1, S19
Serverless — network isolation	Private endpoints in and out, VNet integration, public access disabled	§4.2, S18, S20

6. Producing live evidence (next step)

Part	How to run it for real	Estimated cost
VM	Upgrade to Pay-As-You-Go, or use a region/size the subscription allows; run `vm_hardening/scripts/run_part1.sh`, then `delete_part1.sh`	≈ $1–2.50 per day while it exists
Containers	Install Docker + Trivy on the UTM Ubuntu VM (192.168.64.6); run `container/build_and_scan.sh` and `run_secure.sh`	$0 (local)
Serverless	`az deployment group create -f serverless/infra/main.bicep`; deploy the code with `func azure functionapp publish`	Cents (Flex Consumption free grant; private endpoints ≈ $0.01/h each)

Known limitations of the simulation: figure values (package counts, vulnerability totals, timestamps, Lynis scores and resource IDs) are representative, not measured; Trivy results change daily as new CVEs are published. Live runs replace the figures one for one, because each figure is produced by the exact command shown in it.